

CST8507: Natural Language Processing

Lab 2: Applying Regular Expressions to NLP Tasks

Objectives

- Understand and apply regular expressions to analyze text data.
- Design and implement a rule-based NLP chatbot (ELIZA style).
- Practice text preprocessing.
- Perform a basic sentiment analysis on real-world text data.

Learning Resources

Lecture Slides and resources including Hybrid work (week 2 & 3).

Part 1: Rule-based NLP chatbot (ELIZA)

Background

ELIZA is one of the earliest chatbots, created in 1966 by Joseph Weizenbaum at MIT. It was designed to simulate a psychotherapist by using simple rule-based pattern matching, not real understanding.

ELIZA works by looking for keywords or patterns in the user's input and then applying simple IF-THEN rules to decide how to respond. It generates replies using predefined templates and often reflects the user's own words back as questions. Although this can make the conversation seem meaningful, ELIZA does not actually understand the meaning, emotions, or context of what the user says.

Task Description

Your task is to design and implement a simple ELIZA chatbot that uses a set of predefined rules to analyze user input, match patterns using regular expressions, and generate appropriate responses based on those rules.

The system is working as follows:

The ELIZA chatbot system works by first receiving a user's text input, which is then normalized by converting it to lowercase and removing extra whitespace. The system evaluates the input against a predefined list of rules in a fixed priority order. For each rule, it applies a regular expression to check for a match. If a match is found, the system extracts any captured groups from the input and applies necessary transformations, such as swapping pronouns (for example, "I" becomes "you" and "my" becomes "your"), to make the response coherent. It then selects a response template associated with the rule and substitutes the transformed text into the template. Once a response is generated, rule evaluation stops. If none of the rules match the input, the system applies a fallback rule to return a generic default response.

Rules

Rule Group 1: Greetings

Rule 1

IF the user input contains:

hello | hi | hey

THEN respond with one of:

"Hello. How are you feeling today?"

"Hi there. What would you like to talk about?"

"Hello. Please tell me what is on your mind."

Rule Group 2: Feelings (Core ELIZA Behavior)

Rule 2

IF the input matches:

I feel (.*)

THEN respond with:

"Why do you feel <1>?"

"Do you often feel <1>?"

"What makes you feel <1>?"

Rule 3

IF the input matches:

I am (.*)

THEN respond with:

“How long have you been <1>?”

“Why do you think you are <1>?”

“Do you enjoy being <1>?”

Rule Group 3: Family

Rule 4

IF the input contains:

mother | father | parents | sister | brother

THEN respond with:

“Tell me more about your family.”

“How do you feel about your family?”

“Does your family influence you a lot?”

Rule Group 4: Questions to ELIZA

Rule 5

IF the input matches:

why (.*)

THEN respond with:

“Why do you think <1>?”

“What do you think the answer is?”

Rule 6

IF the input matches:

can you (.*)

THEN respond with:

“What makes you think I can <1>?”

“Why do you ask if I can <1>?”

Rule Group 5: Negative Emotions

Rule 7

IF the input contains:

sad | unhappy | depressed | miserable

THEN respond with:

“I’m sorry you feel that way.”

“What do you think is causing this?”

“Do you often feel like this?”

Rule Group 6: Default

Rule 8

IF no other rule matches

THEN respond with:

“Please go on.”

“Tell me more.”

“Can you elaborate on that?”

Pronoun Substitution Rules

Before inserting <1> into a response, apply transformations:

From	To
I	you
my	your
me	you
am	are
you	I
your	my

Requirement

- Use only Python and regular expressions.
- Implement random choice among multiple responses for variety.

Sample Interaction

You: hi

ELIZA: Hi there. What would you like to talk about?

You: i feel sad

ELIZA: What makes you feel sad?

You: i am not feeling well

ELIZA: How long have you been not feeling well?

You: long time

ELIZA: Tell me more.

You: exit

ELIZA: Goodbye!

Save you code in file and name it “lab2_part1”

Part 2: Rule -Based Sentiment Annotation using Regular Expression

Task description

In this task, you will implement a rule-based sentiment annotation system using regular expressions. The goal is to automatically assign sentiment labels to movie reviews based on the presence of predefined positive and negative words.

You will work with the **IMDb movie reviews dataset** provided by the nltk Python package.

First, you will preprocess the text data. Next, you will design a **simple sentiment scoring mechanism** based on a sentiment lexicon that you define.

For each review, you will count how many positive and negative words appear in the text using pattern matching techniques. Based on these counts, you will assign a sentiment label: *positive*, *negative*, or *neutral*.

Dataset

You will use the **IMDb movie reviews dataset** available from the **nlTK** Python package. This dataset contains movie reviews labeled as either **positive** or **negative**. Each review is raw text and may include punctuation, HTML tags, or URLs.

Data Preprocessing

For this step, you may use **NLTK** or **spaCy** to reprocess the data. Your preprocessing should include:

- Converting all text to lowercase.
- Removing HTML tags (e.g.,
).
- Removing URLs and web links.
- Removing punctuation, numbers, and special characters.
- Removing extra whitespace.

Sentiment Annotation and Evaluation

Analyze each preprocessed review by counting the number of positive and negative words from your sentiment lexicon. Assign a label to the review based on which count is higher: pos for positive, neg for negative, or neutral if they are equal or no words match. This simple annotation method will allow you to evaluate the sentiment of the text before comparing it with the true labels.

The sentiment annotation step should include:

- Define your set of sentiment **lexicon** with positive and negative words (e.g., “good”, “bad”, “excellent”, “terrible”).
- For each review, count how many positive and negative words appear.
- Assign a sentiment label:
 - pos if positive words > negative words
 - neg if negative words > positive words
 - neutral if counts are equal or no words match
- Apply your sentiment annotation on the dataset.
- Calculate the accuracy of your predictions against the true labels.
- Display a sample review showing:
 - Original text
 - Preprocessed text

- True label
- Predicted label

Sample output:

Accuracy on 200 reviews: 0.56

review (original): my giant begins with a monologue that ' s more funny than not and a distinctive ` princess bride, ' medieval fairy - tale - in - the - ? 90s feel. i was pleasantly surprised how sharp the comedy was , with very funny scenes occurring on ...

Preprocessed review: my giant begins with a monologue that s more funny than not and a distinctive princess bride medieva ...

Original label: neg
Predicted label: pos

Requirements

A starter Python script has been provided to help you get started with this lab. You **should use this code as a foundation**, complete the preprocessing, implement the sentiment scoring, and perform the evaluation as described in the assignment tasks.

Save your code in file and name it “lab2_part2”.

Code design and style Requirements.

Make sure that your program is properly documented:

- You should have a docstring at the very beginning of the file briefly describing your program and stating your name, section and creativity additions.
- Each function should have an appropriate docstring (including arguments and return value if applicable).
- Other miscellaneous comments to make things clear.

In addition, make sure that you’ve used good code design and style (including meaningful variable names, constants where relevant, vertical white space, etc.).

Submission Instruction

Submit your code **as a Jupyter Notebook with the running code**, you can do the following steps:

1. Open your Jupyter Notebook: You can open Jupyter Notebook either from the command line by typing "jupyter notebook" or from Anaconda Navigator.
2. Write your code: Write the code you want to submit in the cells of the Jupyter Notebook.
Make sure to run the cells so that the output is generated.
3. Save the Notebook: Once you have written and run your code, save the Jupyter Notebook by clicking on File -> Save and Checkpoint or by pressing "Ctrl + S".
4. Export the Notebook: To export the Jupyter Notebook, go to File -> Download as -> Notebook (.ipynb). This will download the Notebook as a .ipynb file on your laptop

Important Information

For your assignment submission, follow these steps:

1. Create a new folder named lab2
2. Place the following files inside the lab2 folder:
 - lab2_part1
 - lab2_part2
3. Compress the entire lab2 folder into a zip file

Submit your program via Brightspace. Your program file must be named "lab2" You can submit multiple times, with only the most recent submission (before the due date) graded.

Grading Criteria (Part 1)

Criterion	Marks
All rules implemented correctly. Patterns match input, and responses follow templates	20
Transformation / Pronoun Handling	10
Regular expressions correctly match user input.	10
Default / Fallback Handling	5
Code is organized in functions, well commented, and easy to read.	5

Grading Criteria (Part 2)

Criterion	Marks
Data Preprocessing	10
Sentiment annotation	20
Evaluation & Accuracy	10
Output format	5
Code is organized in functions, well commented, and easy to read.	5

Due date

Check Brightspace